싱가포르 1공정 크롤링 보고서

완전 코딩 가이드라인 v3 — 팀 헌법 v2 준수 버전

헌법 준수 변경사항 요약

이 보고서는 팀원 온보딩 가이드(팀 작업 헌법 v2)를 기준으로 싱가포르 1공정 크롤링 로직을 전면 재정비한 최종본입니다. 이전 v10 대비 핵심 변경사항은 아래와 같습니다.

구분	v10 (구)	v3 (헌법 준수)
DB 테이블	singapore (전용)	products (공통 단일)
스키마	sg_ 전용 6컬럼	헌법 필수 9컬럼 + sg_ 확장
크롤러 인터페이스	자체 INSERT 방식	map_to_schema() 표준 함수
설정 파일	없음	sources.yaml (신규)
공통 모듈	미사용	inn_normalizer, normalizer 연결
FOB 계산	1공정 직접 산출	price_local만 저장 → 2공정 위임
cron 스케줄	없음	23:00 UTC (헌법 지정 슬롯)

0. 8 개 품목 최종 분류

HSA PRISM 확인일: 2026-04-11

타입	품목명	성분	비고
타입 A	**정규 등재 · 표준 크롤링**		
	Hydrine	Hydroxyurea 500mg	SIN11083P 직접 추적
	Gadvoa Inj.	Gadobutrol 604.72mg	GADOVIST 레퍼런스
	Sereterol Activair	Fluticasone+Salmeterol	SERETIDE 23건
	Omethyl/Cutielet	Omega-3-EE90 2g	Omacor 1000mg 환산
타입 B	**복합제 · 성분별 수집 후 ×0.85 계산**		
	Rosumeg Combigel	Rosuvastatin+Omega-3	성분별 55건/8건
	Atmeg Combigel	Atorvastatin+Omega-3	성분별 54건/8건
타입 C	**HSA 미등재 · SAR 확장 크롤링**		
	Ciloduo	Cilostazol+Rosuvastatin	SAR + 인근국 루트
	Gastiin CR	Mosapride 15mg	SAR + 인근국 루트

1. sources.yaml (신규 — 헌법 필수)

헌법이 요구하는 3가지 중 첫 번째. team-kit/template/sources.yaml 기반으로 싱가포르 전용 설정을 작성합니다.

🔧 **헌법 수정 적용: sources.yaml 신규 작성**

```yaml
country: "SG"          # ISO 3166-1 alpha-2
currency: "SGD"        # ISO 4217

sources:
  regulator_api:
    tier: 1
    url_seed: "https://hsa.gov.sg/api/..."
    auth: none
    rate_limit_qps: 2.0
    cron_slot: "0 23 * * *"   # 싱가포르 지정 슬롯 (UTC 23:00)

  ndf_api:
    tier: 1
    url_seed: "https://ndf.gov.sg/api/pharmclass"
    auth: none
    rate_limit_qps: 1.0
    cron_slot: "5 23 * * *"

  moh_drug_prices:
    tier: 2
    url_seed: "https://www.moh.gov.sg/cost-financing/drug-prices"
    access_method: html_static
    rate_limit_qps: 0.5
    cron_slot: "10 23 * * *"

  moh_pdf:
    tier: 2
    url_seed: "https://www.moh.gov.sg/newsroom"
    access_method: pdf
    cron_slot: "0 0 * * *"   # 다음날 00:00 UTC

  gebiz_weekly:
    tier: 1
    url_seed: "https://www.gebiz.gov.sg"
    access_method: playwright
    cron_slot: "0 4 * * 1"   # 매주 월요일 04:00 UTC

  guardian_sg:
```

```
    tier: 3
    url_seed: "https://www.guardian.com.sg"
    access_method: playwright
    proxy: residential
    cron_slot: "30 23 * * *"

  watsons_sg:
    tier: 3
    url_seed: "https://www.watsons.com.sg"
    access_method: playwright
    proxy: residential
    cron_slot: "45 23 * * *"
```

---

2. DB 스키마 — products 공통 테이블 (헌법 준수)

🔗 **헌법 수정 적용: singapore 전용 테이블 폐기 → 공통 products 테이블로 통합**

⚠ fob_estimated_usd는 1공정에서 직접 계산하지 않음. price_local(현지 통화)만 저장하고 2공정 FOB 역산 모듈에 위임.

2-1. 헌법 필수 컬럼 (null 불가)

컬럼	타입	싱가포르 적용 예시
country	text	"SG"
currency	text	"SGD"
product_id	text	"SG_hydrine_hydroxyurea_500"
trade_name	text	"Hydrine"
market_segment	text	tender / retail / wholesale / combo_drug
confidence	numeric	0.90~0.94 (1.00 금지)
source_url	text	원본 수집 URL
source_tier	int	1~5 (sources.yaml tier 값)
source_name	text	"hsa_api", "ndf_api", "gebiz_crawler"

2-2. 있으면 채우는 컬럼

컬럼	싱가포르 적용 설명
regulatory_id	HSA 등록번호. 예: SIN11083P (Hydrine)
scientific_name	API 응답의 active_ingredient 그대로
strength	"500 mg", "604.72 mg" 등
dosage_form	"tablet", "injection", "capsule"
price_local	SGD 현지가 (USD 아님). FOB는 2공정에서
atc_code	B01AC23 (Cilostazol), A03FA05 (Mosapride) 등
raw_payload	API 원본 JSON. sg_ 전용 필드 여기에 저장

2-3. 싱가포르 전용 필드 — raw_payload 에 저장

헌법에 따라 나라 전용 필드는 raw_payload 컬럼(JSONB)에 저장합니다.

raw_payload 키	의미
sg_source_type	api_realtime / dynamic_crawl / static_fallback / sar_reference / ai_discovered
sg_gebiz_award	GeBIZ 낙찰 번호 (tender_only 전용)
sg_ndf_listed	NDF 수재 여부 (boolean)
combo_multiplier	복합제 FOB 보정 계수 0.85 (타입 B/C)
outlier_flagged	IQR 이상치 판정 (ESCoE TR-12)
sar_feasibility	confirmed / possible / unlikely (타입 C)
reference_country	인근국 참조 시 해당 국가명

2-4. 자동 채워지는 컬럼 (건드리지 말 것)

컬럼	처리 주체
id	DB가 UUID 자동생성
crawled_at	DB가 now() 자동 기록
inn_name, inn_id, inn_match_type	inn_normalizer.py가 자동 처리
fob_estimated_usd	2공정 FOB 역산 모듈이 나중에 산출

3. 크롤러 공통 인터페이스 — map_to_schema() (헌법 필수)

🔗 **헌법 수정 적용: map_to_schema() 표준 함수 신규 도입 — 모든 sg_*.py 크롤러에 적용**

헌법이 요구하는 3가지 중 두 번째. 각 크롤러의 핵심은 map_to_schema() 함수 하나입니다. 자국 API 응답을 공통 스키마로 변환하면 공통 파이프라인이 자동 처리합니다.

3-1. 공통 헤더 (모든 sg_*.py 크롤러 상단)

```
COUNTRY = "SG"
CURRENCY = "SGD"

def map_to_schema(item, *, source_url, source_name, source_tier):
    return {
        # ── 헌법 필수 9컬럼 ──────────────────────────────
        "country":        COUNTRY,
        "currency":       CURRENCY,
        "product_id":     f"SG_{item.get('reg_no') or item.get('product_id')}",
        "trade_name":     item.get("product_name") or item.get("trade_name"),
        "market_segment": item.get("market_segment", "retail"),
        "confidence":     item.get("confidence", 0.90),  # 1.00 절대 금지
        "source_url":     source_url,
        "source_tier":    source_tier,
        "source_name":    source_name,
        # ── 선택 컬럼 ────────────────────────────────────
        "regulatory_id":  item.get("reg_no"),
        "scientific_name":item.get("active_ingredient"),
        "strength":       item.get("strength"),
        "dosage_form":    item.get("dosage_form"),
        "price_local":    item.get("price"),  # SGD 현지가. FOB는 2공정
        "atc_code":       item.get("atc_code"),
        # ── SG 전용 → raw_payload에 ──────────────────────
        "raw_payload": {
            "sg_source_type":    item.get("sg_source_type", "api_realtime"),
            "sg_gebiz_award":    item.get("sg_gebiz_award"),
            "sg_ndf_listed":     item.get("sg_ndf_listed", False),
            "combo_multiplier":  item.get("combo_multiplier"),
            "outlier_flagged":   item.get("outlier_flagged", False),
            "sar_feasibility":   item.get("sar_feasibility"),
            "reference_country": item.get("reference_country"),
        }
    }
```

```
# ── 공통 파이프라인 (건드리지 말 것) ──────────────────────────
# record = map_to_schema(item, source_url=url, source_name=SOURCE_NAME,
source_tier=1)
# record = normalize_record(record)        # 함량/제형 정규화
# record = _inn.normalize_record(record)  # WHO INN 매칭 자동
# sb.table("products").upsert(record, on_conflict="product_id").execute()
```

---

4. 브랜드 → INN 매핑 (헌법 필수)

🔗 헌법 수정 적용: inn_normalizer.py 연결 — v10에서 누락된 WHO INN 자동 매칭 추가

헌법이 요구하는 3가지 중 세 번째. 싱가포르 현지 브랜드명을 INN에 등록합니다. 등록 후 inn_normalizer.py가 자동으로 inn_name, inn_id, inn_match_type을 채웁니다.

```
# crawlers/sg_inn_map.py  (최초 1회 실행)
from inn_normalizer import _inn

# 싱가포르 전용 브랜드 등록
_inn._brand_map["Hydrine"]           = "hydroxyurea"
_inn._brand_map["Gadvoa"]            = "gadobutrol"
_inn._brand_map["Sereterol"]         = "fluticasone/salmeterol"
_inn._brand_map["Sereterol Activair"]= "fluticasone/salmeterol"
_inn._brand_map["Omethyl"]           = "omega-3 acid ethyl esters"
_inn._brand_map["Cutielet"]          = "omega-3 acid ethyl esters"
_inn._brand_map["Rosumeg"]           = "rosuvastatin/omega-3 acid ethyl esters"
_inn._brand_map["Rosumeg Combigel"]  = "rosuvastatin/omega-3 acid ethyl esters"
_inn._brand_map["Atmeg"]             = "atorvastatin/omega-3 acid ethyl esters"
_inn._brand_map["Atmeg Combigel"]    = "atorvastatin/omega-3 acid ethyl esters"
_inn._brand_map["Ciloduo"]           = "cilostazol/rosuvastatin"
_inn._brand_map["Gastiin"]           = "mosapride"
_inn._brand_map["Gastiin CR"]        = "mosapride"

# 글로벌 브랜드 (이미 inn_normalizer에 내장됨 — 추가 불필요)
# GADOVIST → gadobutrol, SERETIDE → fluticasone/salmeterol
# Omacor → omega-3 acid ethyl esters
```

---

5. 전체 아키텍처 (헌법 준수)

```
┌──────────────────────────────────────────────────────────────┐
│              LOGIC A — 지정 사이트 크롤링                      │
│   (8개 품목 전용, sources.yaml 기반 cron 23:00 UTC)           │
│                                                              │
│  sg_api_light.py       Easy   : HSA API · NDF · MOH · MedPath│
│  sg_dynamic_mid.py     Medium: MOH PDF · Customs · InfoSearch│
│  sg_playwright_heavy.py Hard : Guardian · Watsons            │
│  sg_gebiz_weekly.py    V.Hard: GeBIZ (Gadvoa · Hydrine)      │
│  sg_sar_extended.py    C-Type: Ciloduo · Gastiin SAR 3레이어 │
│                                                              │
│  공통: map_to_schema() → normalize_record() → inn_normalizer│
│        → products 테이블 upsert (product_id 충돌시 갱신)      │
└──────────────────────┬───────────────────────────────────────┘
                       │ Logic A 실패 or 신규 품목
                       ▼
┌──────────────────────────────────────────────────────────────┐
│              LOGIC B — AI 자율 크롤링                          │
│  sg_ai_discovery.py, 계층적 멀티에이전트                       │
│                                                              │
│  검색 에이전트 → 신뢰 URL 탐색 · 도메인 검증                   │
│  분석 에이전트 → 가격 추출 · map_to_schema() 변환              │
│                confidence 상한: 0.75 (ai_discovered)          │
└──────────────────────┬───────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│     utils/iqr.py → products 테이블 → 2공정 FOB 역산           │
│     (fob_estimated_usd는 여기서 처음 계산됨)                   │
└──────────────────────────────────────────────────────────────┘
```

---

6. 프로젝트 폴더 구조

🔗 **헌법 수정 적용: sources.yaml 신규 추가 / sg_inn_map.py 신규 추가**

```
1st_logic/
│
├── .env                            # API 키 (gitignore)
├── requirements.txt
```

```
├── sources.yaml                    # ★ 헌법 필수 (신규)
│
├── crawlers/
│   ├── sg_api_light.py             # Easy  : HSA·NDF·MOH·MedPath
│   ├── sg_dynamic_mid.py           # Medium: MOH PDF·Customs·InfoSearch
│   ├── sg_playwright_heavy.py      # Hard  : Guardian·Watsons
│   ├── sg_gebiz_weekly.py          # V.Hard: GeBIZ (Gadvoa·Hydrine)
│   ├── sg_sar_extended.py          # C-Type: Ciloduo·Gastiin SAR
│   ├── sg_ai_discovery.py          # Logic B: 자율 탐색
│   └── sg_inn_map.py               # ★ INN 브랜드 매핑 (신규)
│
├── utils/
│   ├── db.py                       # Supabase products 테이블 래퍼
│   ├── hsa_registry.py             # HSA CSV 로더
│   ├── iqr.py                      # IQR 이상치 (ESCoE TR-12)
│   ├── combo.py                    # 복합제 price_local 성분 합산
│   ├── normalizer.py               # 공통 normalize_record()
│   └── domain_validator.py         # 신뢰 도메인 필터 (Logic B)
│
├── datas/
│   ├── ListingofRegisteredTherapeuticProducts.csv
│   └── static/products_seed.jsonl
│
├── reports/
├── templates/report_template.docx
└── report_generator.py
```

---

7. 크롤러별 구현 세부사항

7-1. Phase 0 — 환경 세팅

.env

```
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_KEY=eyJ...
CLAUDE_API_KEY=sk-ant-...
PERPLEXITY_API_KEY=pplx-...
SCRAPER_API_KEY=...
HSA_CSV_PATH=datas/ListingofRegisteredTherapeuticProducts.csv
```

requirements.txt

```
httpx[http2]>=0.27
playwright>=1.44
pdfplumber>=0.11
beautifulsoup4>=4.12
supabase>=2.4
tenacity>=8.3
python-dotenv>=1.0
anthropic>=0.28
```

7-2. Phase 1 — DB 시딩 (products 테이블)

🔧 **헌법 수정 적용: products_seed.jsonl 헌법 필수 9컬럼 형식으로 재작성**


```
{"country":"SG","currency":"SGD","product_id":"SG_omethyl_omega3_2g",
 "trade_name":"Omethyl","market_segment":"retail","confidence":0.70,
 "source_url":"","source_tier":2,"source_name":"moh_drug_prices",
 "raw_payload":{"sg_ndf_listed":false,"combo_multiplier":null}}

{"country":"SG","currency":"SGD","product_id":"SG_gadvoa_gadobutrol_604",
 "trade_name":"Gadvoa Inj.","market_segment":"tender","confidence":0.85,
 "source_url":"","source_tier":1,"source_name":"gebiz_crawler",
 "raw_payload":{"sg_ndf_listed":false,"combo_multiplier":null}}

{"country":"SG","currency":"SGD","product_id":"SG_hydrine_hydroxyurea_500",
 "trade_name":"Hydrine","market_segment":"tender","confidence":0.90,
 "source_url":"","source_tier":1,"source_name":"ndf_api",
 "raw_payload":{"sg_ndf_listed":true,"sg_gebiz_award":null}}

{"country":"SG","currency":"SGD","product_id":"SG_ciloduo_cilosta_rosuva",
 "trade_name":"Ciloduo","market_segment":"wholesale","confidence":0.35,
 "source_url":"","source_tier":4,"source_name":"sar_reference",
 "raw_payload":{"sg_ndf_listed":false,"sar_feasibility":"possible"}}
```


7-3. Phase 2 — LOGIC A : Easy 크롤러 (D2)

crawlers/sg_api_light.py — data.gov.sg · NDF · MOH · MedPath

✓ map_to_schema() 적용 후 공통 파이프라인(normalize_record → inn_normalizer → upsert)으로 연결.

```
COUNTRY, CURRENCY = "SG", "SGD"
SOURCE_NAME = "hsa_api"

from tenacity import retry, stop_after_attempt, wait_exponential
import httpx
```

```
@retry(wait=wait_exponential(min=2, max=30), stop=stop_after_attempt(3))
async def fetch_api(url, params=None):
    async with httpx.AsyncClient(http2=True, timeout=20) as client:
        r = await client.get(url, params=params)
        r.raise_for_status()
        return r.json()

def map_to_schema(item, *, source_url):
    return {
        "country": COUNTRY, "currency": CURRENCY,
        "product_id": f"SG_{item['reg_no']}",
        "trade_name": item["product_name"],
        "market_segment": item.get("segment", "retail"),
        "confidence": 0.92,
        "source_url": source_url,
        "source_tier": 1,
        "source_name": SOURCE_NAME,
        "regulatory_id": item.get("reg_no"),
        "scientific_name": item.get("active_ingredient"),
        "strength": item.get("strength"),
        "dosage_form": item.get("dosage_form"),
        "price_local": item.get("price_sgd"),  # SGD. FOB는 2공정
        "raw_payload": {
            "sg_source_type": "api_realtime",
            "sg_ndf_listed": item.get("ndf_listed", False),
            "outlier_flagged": False,
        }
    }
```

7-4. Phase 3 — LOGIC A : Medium 크롤러 (D2~D3)

crawlers/sg_dynamic_mid.py — MOH PDF · Customs · HSA InfoSearch

MOH Newsroom PDF 파이프라인: pdfplumber → Claude Haiku JSON 정형화 (XGrammar 스키마 준거)

```
SG_SCHEMA = {
    "product_id": "string",
    "price_sgd": "number",        # 현지 통화 (SGD), FOB 아님
    "unit": "string",
    "source_date": "YYYY-MM-DD",
    "confidence_note": "string"
}

HAIKU_PROMPT = f"""
```

```
텍스트에서 의약품 가격 정보를 추출하라.
반드시 아래 JSON 스키마만 반환. 다른 텍스트 일절 금지.
price는 SGD 현지가만 기록. USD 환산 금지.
스키마: {json.dumps(SG_SCHEMA)}
"""

def map_to_schema(item, *, source_url):
    return {
        "country": "SG", "currency": "SGD",
        "product_id": f"SG_{item['product_id']}",
        "trade_name": item.get("trade_name", item["product_id"]),
        "market_segment": "retail",
        "confidence": 0.75,
        "source_url": source_url,
        "source_tier": 2,
        "source_name": "moh_pdf",
        "price_local": item.get("price_sgd"),
        "raw_payload": {"sg_source_type": "dynamic_crawl",
                        "outlier_flagged": False}
    }
```

7-5. Phase 4 — LOGIC A : Hard 크롤러 (D3~D4)

crawlers/sg_playwright_heavy.py — Guardian SG · Watsons SG

대상: Omethyl · Sereterol · Rosuvastatin 성분가 · Atorvastatin 성분가

방식: mitmproxy 모바일 API 역추적 → 실패 시 Playwright + Residential Proxy

항목	내용
실패 시	Proxy IP 교체 → 3회 재시도 → MedPath fallback 자동 전환
CAPTCHA	자동 우회(2captcha) 금지. CAPTCHA 발생 시 Render 서버 전환
출력 컬럼	price_local(SGD) 저장. combo_multiplier는 raw_payload에
confidence	dynamic_crawl: 0.70~0.84 (source_tier: 3)

7-6. 복합제 price_local 합산 (combo.py 수정)

🔧 **헌법 수정 적용: FOB 직접 계산 제거 → price_local 성분 합산 후 2공정에 위임**

```
# utils/combo.py — 헌법 준수 버전
# FOB 계산 제거. price_local(SGD) 합산 + combo_multiplier만 기록
```

```
def calc_combo_price_local(product_id, components):
    from utils.iqr import moving_average_30d
    prices = []
    for comp in components:
        p = moving_average_30d(comp)  # SGD 30일 이동평균
        if p is None:
            return {"price_local": None, "outlier_flagged": True}
        prices.append(p)

    price_local = round(sum(prices), 4)  # SGD 합산 (배율 미적용)

    return {
        "product_id": product_id,
        "price_local": price_local,   # 2공정에서 × 0.85 후 FOB 역산
        "raw_payload": {
            "combo_multiplier": 0.85,  # 2공정 참조용 메타
            "sg_source_type":   "dynamic_crawl",
            "outlier_flagged":  False,
        }
    }

# 적용 품목
# Sereterol  : calc_combo_price_local('SG_sereterol', ['fluticasone','salmeterol'])
# Rosumeg    : calc_combo_price_local('SG_rosumeg',
['rosuvastatin','omega3_ee90'])
# Atmeg      : calc_combo_price_local('SG_atmeg',
['atorvastatin','omega3_ee90'])
```

7-7. Phase 5 — LOGIC A : GeBIZ Very Hard (D5~D6)

crawlers/sg_gebiz_weekly.py — Gadvoa (GADOVIST 낙찰가) · Hydrine (SIN11083P 조달가)

실행 주기: 매주 월요일 (sources.yaml cron_slot: "0 4 * * 1")

구현 단계	내용
① session_cache.py	Supabase 암호화 쿠키 저장/로드
② ttl_guard()	만료 5분 전 선제 갱신
③ click_jitter()	피츠 법칙 마우스 궤적 ±15%
실패 시	ttl_guard 실패 → cold start 1회 → 2회 재시도 → PagerDuty 알림

구현 단계	내용
출력	price_local(SGD) + raw_payload.sg_gebiz_award 저장

7-8. Phase 5 — SAR 확장 크롤러 (D4~D5)

crawlers/sg_sar_extended.py — Ciloduo · Gastiin CR 전용 3단 레이어

레이어	내용
레이어 1	HSA SAR 공식 페이지 → SAR 가능 여부 판단 (Claude Haiku)
레이어 2	인근국 공공 DB → Perplexity Sonar + domain_validator (.gov/.edu/who.int)
레이어 3	WHO/HAI 국제 약가 → ATC 코드 기반 참고가 (레이어 2 전부 실패 시)
출력	price_local 저장 불가 시 raw_payload.sar_feasibility = 'possible' 기록
confidence	sar_reference: 0.35 / regional_reference: 0.35 / static_fallback: 0.30

---

8. LOGIC B — AI 자율 크롤링 (D5~D6)

crawlers/sg_ai_discovery.py — WebVoyager ACL 2024 + arXiv:2508.16571 계층적 에이전트

8-1. 발동 조건

- Logic A 전체 실패 (특정 품목 price_local = NULL)
- 8 개 품목 외 신규 약품 조회 요청
- 수동 호출: python crawlers/sg_ai_discovery.py --product "신규품목"

> 🔧 **헌법 수정 적용: 분석 에이전트 출력도 map_to_schema() 경유 → products 테이블 upsert**

8-2. 분석 에이전트 map_to_schema 변환

```
def analysis_agent_to_schema(product_id, ai_result, source_url):
    """Logic B 결과를 헌법 공통 스키마로 변환"""
    return {
```

```
        "country": "SG", "currency": "SGD",
        "product_id": f"SG_{product_id}",
        "trade_name": ai_result.get("trade_name", product_id),
        "market_segment": ai_result.get("market_segment", "retail"),
        "confidence": min(ai_result.get("confidence", 0.50), 0.75),  # 상한 0.75
        "source_url": source_url,
        "source_tier": 5,            # AI 자율탐색은 tier 5
        "source_name": "ai_discovery",
        "price_local": ai_result.get("price_local"),  # SGD. FOB는 2공정
        "raw_payload": {
            "sg_source_type": "ai_discovered",
            "outlier_flagged": ai_result.get("outlier_flagged", False),
            "discovery_query": ai_result.get("discovery_query"),
        }
    }

# Atomic Calibration (arXiv:2403.14151): confidence 동적 산출
def _calibrate_confidence(record, trusted_domains):
    score = 0.50
    if record.get("price_local"):              score += 0.05
    if record.get("source_url"):               score += 0.05
    if is_trusted_domain(record["source_url"], trusted_domains): score += 0.10
    if record.get("raw_payload", {}).get("discovery_query"): score += 0.05
    return min(round(score, 3), 0.75)  # ai_discovered 상한 0.75
```

8-3. Logic B 호출 예시

```
# Logic A 실패 시 자동 fallback
python crawlers/sg_ai_discovery.py \
  --product "omethyl_omega3_2g" \
  --active "omega-3 ethyl esters" \
  --name "Omethyl" \
  --trigger "logic_a_fail"

# 신규 품목 추가
python crawlers/sg_ai_discovery.py \
  --product "new_drug_abc" \
  --active "amlodipine" \
  --name "Norvasc" \
  --trigger "new_product"
```

---

9. 공통 유틸리티 모듈

모듈	하는 일	나라 무관 이유
inn_normalizer.py	브랜드→INN 매칭 자동화	WHO INN은 전 세계 동일
normalizer.py	함량/제형/가격 정규화	단위 변환 로직 나라 무관
trafilatura_fallback.py	HTML→텍스트 추출	HTML이면 어디든 작동
backoff_retry.py	HTTP 재시도 (tenacity)	나라 무관
supabase_state.py	서킷브레이커/토큰버킷	products 테이블 공유
iqr.py (SG 전용)	IQR·SAND 이상치 탐지	ESCoE TR-12 준거
domain_validator.py	신뢰 도메인 필터	Logic B 전용

---

10. 전체 실행 흐름 및 Confidence 정책

품목	Logic A 채널	Confidence	Logic B 발동 조건
Hydrine	NDF→MOH→GeBIZ	0.90~0.94	3채널 모두 실패 시
Gadvoa	GeBIZ 전용	0.85~0.89	GeBIZ 세션 실패 시
Sereterol	Guardian→Watsons	0.70~0.84	프록시 3회 실패 시
Omethyl/Cutielet	MOH→Guardian	0.70~0.84	동일
Rosumeg	성분별 합산	0.65~0.74	성분가 수집 실패 시
Atmeg	성분별 합산	0.65~0.74	동일
Ciloduo	SAR 3레이어	0.30~0.39	레이어3도 실패 시
Gastiin CR	SAR 3레이어	0.30~0.39	동일
Logic B 결과	**ai_discovered**	**0.50~0.75**	**Atomic Calibration 동적 산출**

⚠ confidence 1.00은 절대 금지 (헌법 명시). api_realtime 최대값: 0.94

---

11. Day-by-Day 구현 일정

일정	작업 내용
D1 오전	.env 세팅 + requirements 설치 + HSA CSV 로드 + sources.yaml 작성 (헌법 필수)
D1 오후	products 테이블 생성 + seed INSERT(헌법 형식) + sg_inn_map.py 실행
D2	sg_api_light.py (map_to_schema 적용) + utils/iqr.py 기반 구현
D3	sg_dynamic_mid.py (price_local SGD 저장) + combo.py 수정 (FOB 계산 제거)
D4	sg_playwright_heavy.py + sg_sar_extended.py + domain_validator.py
D5	sg_gebiz_weekly.py (session_cache → ttl_guard → click_jitter)
D6	sg_ai_discovery.py (map_to_schema 연결) + Atomic Calibration
D7	통합 테스트: IQR 검증 + inn_normalizer 매칭 확인 + products 테이블 쿼리 검증
D8~D10	report_generator.py 연동 + 8개 품목 보고서 출력 검증 + QA

---

12. 성공 확률 최종

품목	확률	비고
Hydrine	85%	SIN11083P 직접 추적
Gadvoa	65%	GeBIZ 낙찰이력 존재 여부
Sereterol	80%	SERETIDE 23건 레퍼런스
Omethyl/Cutielet	70%	1000mg→2g 환산
Rosumeg	70%	성분 풍부, 합산 로직
Atmeg	70%	동일
Ciloduo	60%	SAR 3레이어 + Logic B
Gastiin CR	55%	인근국(필리핀·인도) 참조
Logic A만 사용	**~35%**	**전체 8개 완료 기준**
Logic B 추가	**~60%**	**현재 구조**

품목	확률	비고
핵심 4개 완료	**~75%**	**Hydrine·Gadvoa·Sereterol·Omethyl**

13. 12 개국 교차비교 쿼리 (헌법 준수 시 자동 동작)

inn_normalizer 연결 후 products 테이블을 공유하면 아래 단일 쿼리로 싱가포르 포함 12개국 동일 INN 가격을 한 번에 조회할 수 있습니다.

```
-- 12개국 동일 INN 가격 교차비교
SELECT country, currency, price_local, fob_estimated_usd
FROM products
WHERE inn_name = 'hydroxyurea'
  AND deleted_at IS NULL
ORDER BY country;

-- 싱가포르 전용 조회
SELECT trade_name, price_local, confidence, source_name,
       raw_payload->>'sg_source_type' as sg_type,
       raw_payload->>'sg_ndf_listed'  as ndf_listed
FROM products
WHERE country = 'SG'
  AND deleted_at IS NULL
ORDER BY confidence DESC;
```

✓ 헌법 준수로 달성되는 핵심 가치: 12개국이 products 단일 테이블에 INSERT하므로 위 쿼리 1개로 전체 국가 교차비교가 즉시 가능합니다.

참고문헌

[1] ESCoE TR-07 — Outlier Detection Methodologies

[2] ESCoE TR-12 — Applying Machine Learning to Detect Outliers in Alternative Data Sources (Mao et al., 2021)

[3] De Jure: Iterative LLM Self-Refinement for Regulatory Document Analysis (arXiv:2604.02276, 2026)

[4] XGrammar: Helping LLMs Follow Structures (Dong et al., MLC AI/CMU, arXiv:2411.15100, 2024)

[5] Atomic Calibration of Low-Confidence Predictions in LLMs (Duan et al., arXiv:2403.14151, 2024)

[6] SAND: Streaming Subsequence Anomaly Detection (Boniol et al., VLDB 2021, arXiv:2104.03154)

[7] WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models (He et al., ACL 2024, arXiv:2401.13919)

[8] LLM-Based Agents for Competitive Landscape Mapping in Drug Asset Due Diligence (arXiv:2508.16571, 2025)

[9] Addressing the Supplier Selection Problem by AHP — ACI Pharma Case Study (Heliyon/PMC, 2023)

[10] OMOP Common Data Model — Expanding OMOP CDM for Hospital Price Transparency (Altosaar et al., OHDSI 2023)